

Weight Isolation-based Binarized Neural Networks Accelerator

Zhangkong Xian
School of Electronic and Information
Engineering
Beihang University
Beijing, China
xianzhangkong@buaa.edu.cn

Hongge Li*
School of Electronic and Information
Engineering
Beihang University
Beijing, China
*honggeli@buaa.edu.cn

Yuliang Li
School of Electronic and Information
Engineering,
Beihang University
Beijing, China
liyuliang@buaa.edu.cn

Abstract—In this paper, we introduce a binary neural network accelerator which using a new binarization method and hardware sparse. We propose the weights and the activations to either 1 or 0 instead of +1 or -1, which makes the convolution process simplified and more suitable for hardware implementation. To decrease the data access from off-chip memory, we propose a novel data reuse method, which can reduce 58.8% data access, while the weight isolation logic is designed to reduce power consumption. Based on the weight isolation and the retiming technique, the proposed BNN accelerator achieves low power consumption at 500MHz clock by the VC709 Evaluation Kit. Experimental results show that the proposed accelerator achieves a throughput of 3378 GOPS and 1624 GOPS/W energy efficiency.

Keywords—binary neural networks, weight isolation, high-efficiency, binarization method, FPGA

I. INTRODUCTION

Convolutional neural networks have demonstrated state-of-the-art results in many fields, such as image classification [1], object detection [2], speech recognition [3], and even artificial intelligence for games against human beings[4]. However, CNN-based systems usually need large amounts of memory and computational power, which makes them often unsuitable for edge computing as cell phones and smart electronics. In order to make convolutional neural networks more suitable for edge computing, convolutional neural network with binary value technology is proposed [5]. By quantizing the network parameters to 1 bit, the requirement of storage space for a network is significantly reduced. For example, compared to 16-bit, 8-bit fixed-point networks, the requirements for storage space of binary networks are reduced to 1/16 and 1/8, respectively.

Moreover, a large number of experiments have shown that binary convolutional neural networks can also achieve state-of-the-art results in classification or other tasks [6-11]. And in the binary convolutional neural network, since the data is binary, bit operations can be used instead of multipliers, which not only speeds up the calculation but also shows significant improvement in energy efficiency.

Due to the resource efficiency and energy efficiency, many hardware architectures for BNNs have been presented. Yaman Umuroglu et al[7] present Finn, a framework for building fast and flexible FPGA accelerator using a flexible heterogeneous streaming architecture, which reaching 1033 GOPS/W high energy efficiency. In the paper [8], a fully binarized neural network accelerator was proposed. And the accelerator can reach a peak throughput of 2.2 TOPS, and 699 GOPS/W of energy efficiency. Thanh Nguyen presented a high-performance streaming architecture, which reaching a throughput of 1.877 TOPS [11].

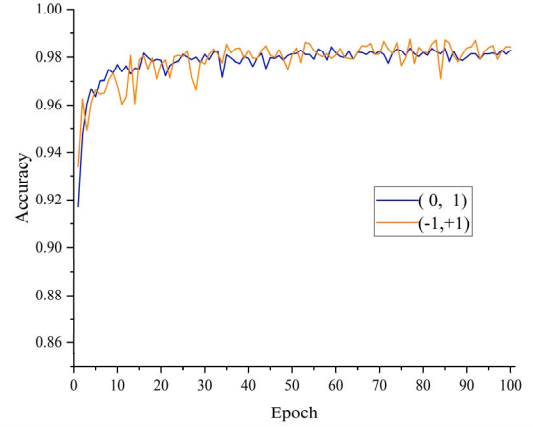


Fig. 1. The classification accuracy of the two binarization methods

In this paper, we focus on comparing the energy efficiency of binarized neural network accelerators, as shown in Fig.1. Although most of the accelerators have achieved a high throughput of TOPS, the power consumption of FPGA-based accelerators is still very high. In this paper, we introduce a BNN accelerator design on FPGA, with a new binarization method and related optimizations to achieve great flexibility, low power and high energy efficiency.

II. BINARIZATION METHOD AND PROCESSING UNIT

A. Binarization Method

The weights and activation of BNN are restricted to 2-values (+1 or -1) is proposed in paper [1]. And in hardware implementations, researchers usually take 2-bits or 1-bit to represent +1 and -1. If only using one bit, researchers should take 0 and 1 as the basic values. It means that BNN method should use 0 to replace -1. This may cause a problem that additional operations are required when getting the final convolution result.

For example, the input image data of the first layer is usually 8-bits, and when the weight is -1, a subtraction operation is required; Even for the hidden layers, both the feature map data and weight based ± 1 applied pop-count operation, then, subtraction and left-shifting operations are needed[10]: $R = 2 \times S_{pop} - V_{vec_len}$, where R is the final convolution result, S_{pop} is the result of pop-count operation and V_{vec_len} is the size of the convolution kernel.

In order to reduce the additional operations and more suitable for hardware implementation, the new binarization method is considered,

$$Binarized(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

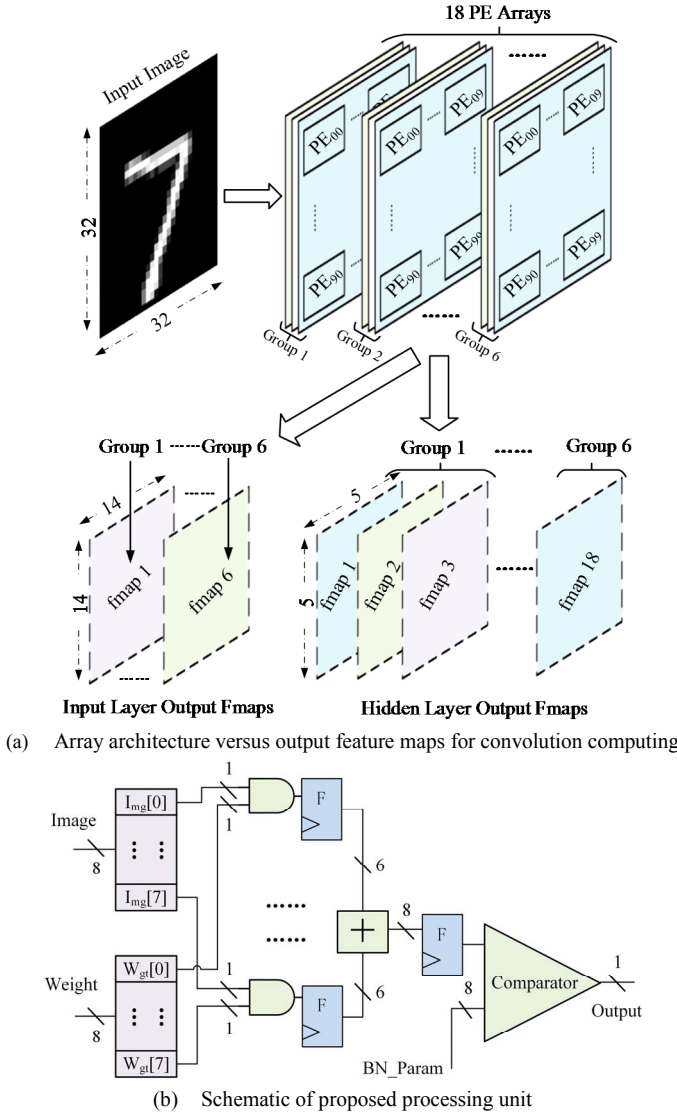


Fig.2. Overview of the proposed accelerator architecture.

where x is the real-valued variable. When training a BNN model, in order to make the parameters update normally during the back propagation, we also using the $\text{Htanh}(x)$ function and the high precision weights as mentioned in [1].

We compare the classification accuracy of the two binarization method on the MNIST dataset. An improved BNN model is used, which including 2 convolution layers, 2 pool layer, 2 normalization layer, and 3 fully connected layers. Two binarization methods are applied in the same model and trained for 100 epochs. The classification accuracy is shown in Fig.1. Although the accuracy and the convergence speed of the two binarization methods are basically the same, the new binarization method fluctuates more gently.

B. Design of the Computing Core

In recently proposed state-of-the-art BNN accelerators, because they constrain both the weights and the activations to either +1 or -1, so the dot product operation can be estimated by XNOR operation. In this paper, since the weights and the activations are constrained to either 0 or 1, then the convolutions can be estimated by AND and pop-count operations.

Fig.2(a) shows the accelerator architecture of array computing, which consists of 18 computational arrays. These computing arrays are reconfigurable to match 6 or 18 output feature maps for different layers. Every PE array be set 10*10 units, so that the convolution operation including 14*14 feature map and 5*5 kernel can be computed in one kernel loop. Here, 32*32 input image is decomposed to nine 14*14 parts, so that one frame input image can be calculated in 9 loops.

Because the weights and the activations are constrained to either 0 or 1, then the convolutions can be estimated by AND logic and pop-count operations. And other operations such as subtraction and left-shifting are not required when the convolution is calculated. Single processing unit structure is shown in Fig.2(b). In order to reduce the time complexity, we unroll the convolution kernel. The input image with 8-bits width is matched by 8 AND gates for convolution operations. Each AND gate followed a pop-count operation to calculate the partial sum. Then, an adder calculates the final convolution result. When the final convolution result is got, the batch norm and binarize operation will be finished through a comparator[9].

In order to reduce path delay and improve performance, we use retiming and pipeline technology to optimize the processing unit. As Fig.2(b) shows, we insert registers after the AND gate and the adder to split the critical path, which enables the accelerator to meet higher timing constraint.

III. DATA REUSE AND WEIGHT CONTROL LOGIC

A. Data Reuse

Convolutional neural networks usually contain a large number of parameters and feature map. During the calculation process, most of data need to be read multiple times. For example, if the stride of the convolution for 5*5

Algorithm 1: Data Reuse

Note:

Clk: Calculation cycle

Ct: Concatenation modules

Ct_num: the number of data concatenation modules

t: for input layer, t is the tile number of input image; for hidden layer, t is the number of input fmaps

k: for input layer, k = kernel size; for hidden layer, k = 1

j: for input layer, j = k*k; for hidden layer, j = [k * k/8]

```

for loop = 1 to t do
  for Clk = 1 to j do
    if Clk <= k then
      for i = 1 to Ct_num do
        Cti read data from DRAM
      end for
    else if Clk > k do
      if i = Ct_num do
        Cti read data from DRAM
      else if i = Ct_num-1 do
        input layer: Cti read data from DRAM
        hidden layer: Cti read data from DRAM and Cti+1
      else do
        for i = 1 to Ct_num-2 do
          input layer: Cti read data from FIFO
          hidden layer: Cti read data from Cti+1, Cti+2
        end for
      end if
    end if
  end for
end for

```

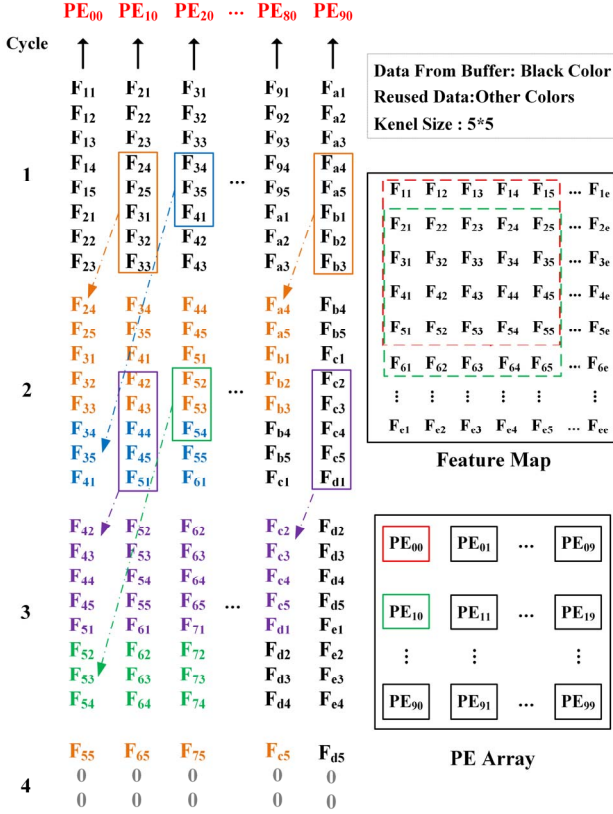


Fig.3. Data flow of the data reuse module

kernel is 1, then 80% of the image data needs to be read repeatedly during the convolution process. If the data cannot be reused, the power consumption of the accelerator will significantly increase.

Algorithm 1 demonstrates the procedure of data reuse in convolution layer. Note that the operations of input layer and hidden layers are different. For input layer, we tile the input image to several parts, each time calculate one of them. If the kernel size is $k \times k$, then we need $k \times k$ clock cycles to finish the calculation of a single loop. During the calculation, we use FIFO to store the data for reuse. During the first k cycles, all the concatenation modules need to read data from DRAM, from $k+1$ cycle, only the last concatenation module need to read data from DRAM, other modules read data from FIFO. For hidden layer, each time calculate only one of the input feature maps. Note that hidden layers do not use FIFO to store the reused data. During the calculation, only the last two concatenation modules read data from DRAM, from second cycle, Ct_1 to Ct_8 read data from Ct_{i+1} and Ct_{i+2} by data concatenation.

To detail data reuse method, Fig.3 shows the data flow of the data reuse module. Here the convolution kernel size is 5×5 , the feature map size is 14×14 , and the PE array is 10×10 . It should be noted that the data in the feature map is binary and each PE is responsible for the calculation of a convolution window. In the data reuse module, the reuse method of each column of the PE array is exactly the same, so we use the data flow of the first column PE to illustrate our data reuse method.

In the first cycle of the convolution process, all the PE needs to read the required data (black data) from the external memory. And then, from the second cycle, only the last two

TABLE I. PERCENTAGE OF 0-WEIGHT

Layer	Conv1	Conv2	FC1	FC2	FC3	Average
Percentage	79.3%	81.1%	79.9%	81.2%	57.3%	75.8%

PEs (PE80, PE90) need to read data from external memory. The remaining PE can obtain this data through data reuse.

Specifically, during the second calculation cycle, the 8 bits data of PE00 can be obtained by concatenating the last 5 bits data in the first cycle of PE10 and the middle 3 bits data in the first cycle of PE20. It is exactly the same for other subsequent calculation cycles. In our array architecture, based on 10×10 PE array, 14×14 binarized feature map and 5×5 convolution kernel size, $10 \times 10 \times (5 \times 5) = 2500$ bits data from off-chip memory should be read without the data reuse. However, using our data reused method, only $8 \times 100 + 10 \times (11 \times 2 + 1) = 1030$ bits data from off-chip memory should be read, which has an 58.8% decrease of frequent accesses to off-chip memory.

B. Weight Isolation Logic

Consider two input vectors $F = \{f_i\}$ and $W = \{w_i\}$, which consist of binarized values either +1 or -1. Then the product of the corresponding elements in the two vectors will also be either +1 or -1, respectively. Means that regardless of the value of the two vectors, their dot product results are valid because they have an effect on the final result. But if the elements of the binarized vectors are 0 or 1, the situation will be different. During a dot product process, only if one of f_i or w_i is 0, the result of dot product must be 0. Then, the number does not need to be transferred to the PE unit, which can reduce the power consumption of data transmission.

In order to reduce invalid data transmission and power consumption during calculation, we propose a weight isolation logic, namely, hardware sparse architecture. The image data is transmitted to the PE array only when the weight is 1, otherwise the new data will be isolated. As shown in Fig.4, we add the weight isolation logic between the data reuse module and the PE array to reduce the transmission of invalid image data. The weight isolation logic receives the weight data from the weight buffer as the control signal. Using the weight isolation technique, we significantly reduce the signal toggle rate and the power consumption during the calculation.

We use python to count the percentage of 0-weight of each layer in the BNN model, as shown in table I. Among two convolution layers, nearly 80% of the weight are 0. For the entire network, there is an average of 75.8% weight are 0, meaning that there is a large number of invalid image data transmission.

IV. HARDWARE IMPLEMENTATION

To verify the accelerator proposed, we use an improved BNN model, which has five layers: two convolution layers and three fully connection layers. Our accelerator is implemented by the VC-709 Evaluation Kit. The resource utilization and power consumption are reported in Vivado Design Suite after implementation. The memory part of this accelerator is realized by the on-chip RAMs. And since we have design high energy efficiency processing unit and array architecture, none of the DSP blocks need to be used.

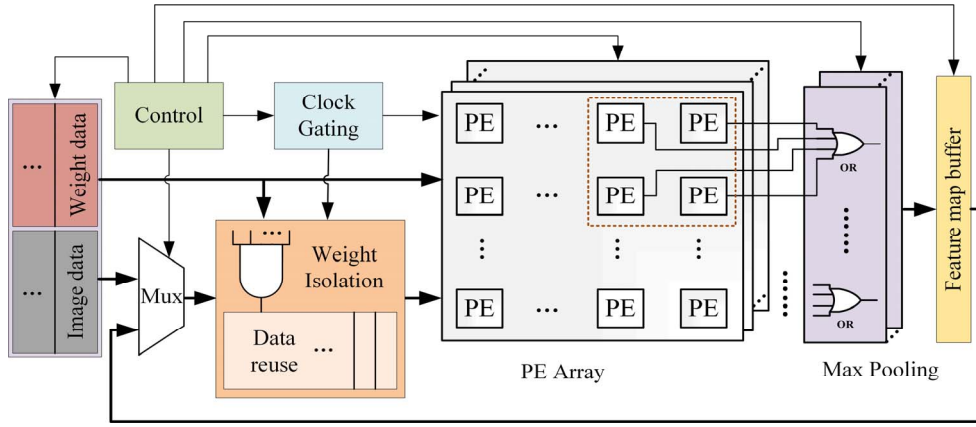


Fig.4. The overall architecture of the accelerator based on the weight-isolation

TABLE II. COMPARISON OF PREVIOUS WORKS

BNN	FPGA 2017[7]	NC 2018[10]	JCSC 2019[8]	VLSI 2019[11]	This work
Platform	ZYNQ	Stratix-V	ZYNQ	Vertex-485T	Vertex-690T
Precision	1bit weight	1bit	1bit	1bit weight	1bit
Dataset	MNIST	MNIST	SVHN	PASCAL VOC	MNIST
Frequency (MHz)	200	150	-	200	500
DSP Used	-	384	-	272	-
BRAM Used (18Kb)	396	2210	103	1214	23
LUT Used	82988	-	29600	104700	65413
FF Used	-	-	-	140100	54891
Performance (GOPS)	9086	12219	2236	4420	3378
Power(W)	8.8	26.2	3.2	14.72	2.08
GOPS/KLUT	109.5	-	75.5	42.2	51.7
Efficiency (GOPS/W)	1033	466	699	300	1624

The overall architecture of the accelerator is shown in Fig.4. To further improve the performance of the accelerator, we also use retiming and pipelining techniques at a higher level. As shown in Figure 4, we logically partitioned between modules, and a sufficient pipeline was implemented between each module. Based on the above optimization, the proposed accelerator can reach up to 500MHz frequency. Although the operating frequency is increased, the accelerator can maintain low power consumption based on the proposed data reuse method and weight control logic.

The throughput of the accelerator can be calculated by the following equation:

$$THP = \frac{\#Ops_per_image}{\#Cycles_per_image} \cdot f$$

where Ops_per_image is the number of total operations, $Cycles_per_image$ is the number of clock cycles, f is the clock frequency.

The comparison between the proposed accelerator and other recent works on BNN hardware implementations is presented in Table II. At 500MHz clock frequency, our design is able to reach a throughput of 3378 GOPS. More importantly, compared to recently proposed state-of-the-art accelerators, our accelerator's energy efficiency achieves 1624 GOPS/W.

V. CONCLUSION

In this paper, the hardware sparse architecture based on the weight isolation is proposed for high efficiency binary neural network. A new algorithm of the input data reuse to decrease the access to off-chip memory is proposed. Since the weights and the activations are constrained to either 0 or 1, the convolutions operation be implemented by AND gate and pop-count operations. The weight isolation and data reuse significantly reduces the frequent accesses rate and the power consumption. Based on the weight isolation and the retiming technique, the proposed BNN accelerator achieves low power consumption at 500MHz clock. The results show that the accelerator achieves high throughput, 58.8% decrease of data access, and 1.624 TOPS/W high energy efficiency based on FPGA.

REFERENCES

- [1] A. Krizhevsky, et al, "ImageNet classification with deep convolutional neural networks," in Proc. Adv. Neural Inf. Process. Syst, pp. 1097–1105, 2012.
- [2] J. Redmon, et al, "YOLO9000: Better, Faster, Stronger," 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, pp. 6517–6525, 2017.
- [3] A. Hannun, Case C, Casper J, et al., "Deep speech: Scaling up end-to-end speech recognition," arXiv:1412.5567, Dec. 2014.
- [4] D. Silver, A. Huang, C.J. Maddison, A. Guez, L et al, Mastering the game of go with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489.
- [5] Courbariaux M et al, "Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1" [J]. 2016, arXiv:1602.02830.
- [6] Rastegari M, et al, "XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks[C] ", European Conference on Computer Vision. 2016.
- [7] Y. Umuroglu, N.J. Fraser, et al, "Finn: a framework for fast, scalable binarized neural network inference", ACM, 2017, pp. 65–74.
- [8] Donglin W,Guo P, et al. A High-Efficiency FPGA-Based Accelerator for Binarized Neural Network[J]. Journal of Circuits, Systems, and Computers,2019,28(1):1
- [9] H. Yonekawa, H. Nakahara, "An On-chip Memory Batch Normalization Free Binarized Convolutional Deep Neural Network on an FPGA", IPDPS, 2017, pp.98-105.
- [10] Shuang Liang, Shouyi Yin, et al, "FP-BNN: Binarized neural network on FPGA, " Neurocomputing, Volume 275, Pages 1072-1086, 2018.
- [11] D. T. Nguyen, et al, "A High-Throughput and Power-Efficient FPGA Implementation of YOLO CNN for Object Detection," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems.2019.